

BHARATI VIDYAPEETH'S COLLEGE OF ENGINEERING, NEW DELHI

Department of Computer Science and Engineering

MICRO PROJECT PROGRESS REPORT

Period of Report from 03.09.2026 to 30.09.2026

Sr. No.	Student Name	Enrollment Number
01	DEV DOGRA	35711515624
02	DHARIYA JALAN	07311515624
03	ADARSH BHARDWAJ	08411515624

Attendance and Feedback By Mentor

Enrollment No.	Student Name	Attendance	Feedback/Suggestion	Signature
35711515624	DEV DOGRA			
07311515624	DHARIYA JALAN			
08411515624	ADARSH BHARDWAJ			

Mentor Name: Dr. Srishti Vashishtha

Approved Project Topic: DataSaathi - Conversational Text-to-SQL with RAG & Visualization

Work done during the period:

1. Literature Review: Surveyed 10+ papers including Seq2SQL (2017), Spider (2018), PICARD (2021), RESDSQL (2023), RAG (Lewis 2020), DIN-SQL (2024), DBCopilot (2024), and Liu et al.'s LLM-for-Text-to-SQL survey (2024). Identified the key gap: no system integrates RAG schema retrieval, multi-level SQL validation, error self-correction, and visualization-as-verification in one pipeline.

2. Architecture Design: Designed a ten-stage pipeline: RAG Retrieval (TF-IDF schema indexing) -> Intent Extraction (analytical type, metric, dimensions, filters) -> SQL Generation (LLM-powered: OpenAI/Groq/Gemini) -> Multi-Level Validation (syntax, schema, safety, semantic) -> Error-Feedback Retry (up to 3 retries) -> Result Verification -> Visualization Planner (rule-based chart mapping with mismatch detection) -> Conversational State (multi-turn context).

3. Backend Implementation (Python / FastAPI):

- rag/retriever.py - Schema document builder + TF-IDF retrieval. Enhanced to inject complete schema overview (all tables, columns, relationships) into LLM context.
- sql/generator.py - LLM-powered SQL generation. Improved SYSTEM_PROMPT for strict SQLite output. Added clean_sql() to strip markdown fences and comments before execution.
- sql/validator.py - Four-level validation: syntax, schema (table/column existence), safety (blocking DELETE/DROP/UPDATE), and semantic verification against intent.
- database/manager.py - Database connection + schema introspection. Added CSV parsing and ingestion into SQLite. Extended use_file_upload() for .sqlite, .db, .csv with auto schema discovery and row/column counting.
- api/db_schemas.py & api/routes.py - Added TableSummary model. Added table preview endpoint, enhanced status endpoint with table summaries and dynamic suggested questions. Extended upload endpoint for CSV alongside SQLite.
- services/pipeline.py - Enhanced orchestrator for user-uploaded databases and multi-table joins. Enabled rule-based fallback for compatible table structures.
- services/llm_provider.py - Upgraded try_parse_json() to extract JSON from markdown blocks and extra model commentary reliably.
- visualization/planner.py - Smart column classification (time vs. metric keywords), numeric detection from data. Auto-selects: line for time-series, bar for categories, horizontal_bar for rankings. Structural mismatch detection for viz-as-verification.
- verification/result_verification.py, conversation/state.py - Result shape verification and multi-turn session state. Configured LLM_PROVIDER=groq (openai/gpt-oss-20b).

4. Frontend Implementation (React + Vite + Recharts):

- ChartView.jsx (NEW) - Multi-chart engine: Bar, Line, Area, Donut, Table. Chart switching, summary stats (Total/Avg/Min/Max), tooltips, custom colors, CSV export. Built with Recharts.
- DatasetExplorer.jsx (NEW) - Sidebar showing database tables with row counts, column badges, and preview buttons.
- DatasetHeroBanner.jsx (NEW) - Header with database metrics and dynamically generated suggested query chips.
- DatabaseConnectModal.jsx (ENHANCED) - Drag-and-drop upload with progress and status feedback. Supports

.sqlite, .db, .csv, and PostgreSQL connections.

- TablePreviewModal.jsx (NEW) - Modal displaying sample rows and column definitions.
- App.jsx (MAJOR REWRITE) - Integrated sidebar + hero banner. Added KPI Metric Cards for single-value results (gradient card with icon, label, formatted value). ChartView for multi-row results with auto chart selection. Toast notifications. Viz mismatch warnings.
- services/api.js - Added getTablePreview() and uploadDatabaseFile() with FormData handling.

Status / Stage of the Project:

The core pipeline (Stage 1 + 2) is fully implemented, tested, and functional. Users can upload SQLite/CSV files via drag-and-drop, browse schema in the sidebar, preview table data, ask natural language questions triggering the full pipeline (RAG -> intent -> SQL -> validation -> execution with retry -> verification -> visualization), view results as auto-selected charts (Bar/Line/Area/Donut) or KPI metric cards, switch chart types, view summary statistics, export data, see the full provenance trail, and ask follow-up questions. Backend (FastAPI:8000) and frontend (Vite/React:5173) are integrated. All 8 integration tests pass. Stage 3-6 (clarification, probing, Power BI, evaluation) are scaffolded as independently testable code.

Next Phase: Advanced Features and Evaluation

We will wire Stage 3 (uncertainty-gated clarification) with the ANSWER / INVESTIGATE / ASK / ABSTAIN decision policy and database probing (safe exploratory queries). We will add calibrated confidence scoring and a formal evaluation harness measuring intent accuracy, SQL execution success, visualization-type match, and latency. The goal is demonstrating that the closed-loop pipeline with visualization-as-verification outperforms open-loop Text-to-SQL systems.

(Signature of Mentor)

(Signature of the Head of the Department)